

Graph Analytics

Modeling Chat Data using a Graph Data Model

Our graph model maps the internal company communications into nodes and relationships to facilitate deep network analysis.

- **Nodes:** We model four core entities: User, Team, TeamChatSession, and ChatItem. Every node has an indexed, unique id property derived exactly from the CSV inputs.
- **Relationships:**
 - (User)-[:CreatesSession]->(TeamChatSession): Tracks which user initiated a chat session.
 - (TeamChatSession)-[:OwnedBy]->(Team): Connects the session to the overarching project team.
 - (User)-[:Joins]->(TeamChatSession) & (User)-[:Leaves]->(TeamChatSession): Tracks user participation timeline within sessions.
 - (User)-[:CreateChat]->(ChatItem): Links a specific message back to its author.
 - (ChatItem)-[:PartOf]->(TeamChatSession): Connects the individual message into the session context.
 - (ChatItem)-[:Mentioned]->(User): Indicates an explicit @mention in the chat, allowing us to build direct interaction graphs.
 - (ChatItem)-[:ResponseTo]->(ChatItem): Maps conversation threads. A newer chat item points back to the older chat item it replies to.
- **Properties:** Every structural edge generated from the raw logs carries a timeStamp property (parsed safely as a float). This allows for temporal sequence reconstruction and ensures data is preserved rather than discarded during import.

Creation of the Graph Database for Chats

Describe the steps you took for creating the graph database. As part of these steps

- i) Write the schema of the 6 CSV files

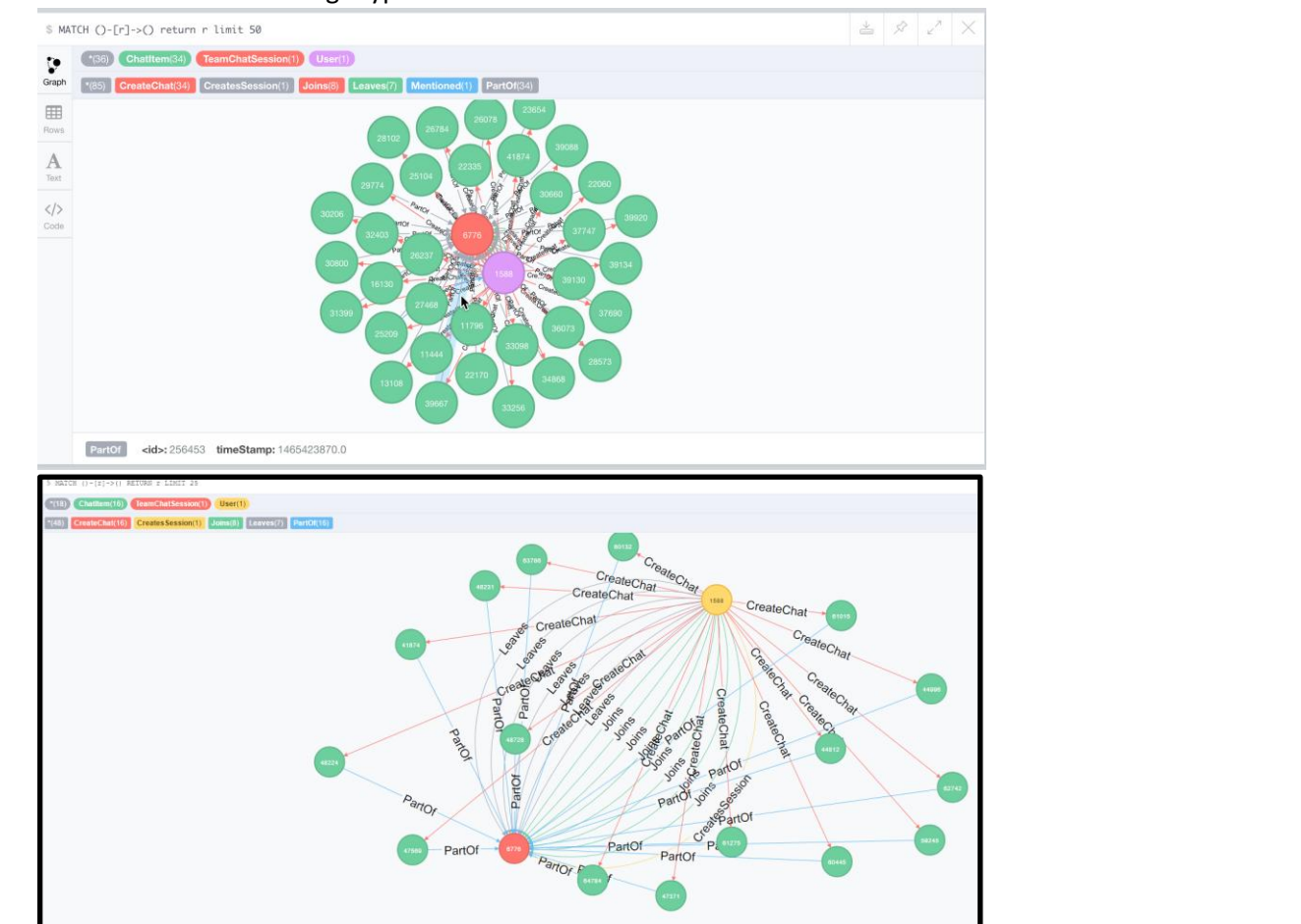
```
CREATE CONSTRAINT ON (u:User) ASSERT u.id IS UNIQUE;
CREATE CONSTRAINT ON (t:Team) ASSERT t.id IS UNIQUE;
CREATE CONSTRAINT ON (c:TeamChatSession) ASSERT c.id IS UNIQUE;
```

```
CREATE CONSTRAINT ON (i:ChatItem) ASSERT i.id IS UNIQUE;
```

- ii) Explain the loading process and include a sample LOAD command

```
LOAD CSV FROM "file:///chat_item_team_chat.csv" AS row
MERGE (u:User {id: toInteger(row[0])})
MERGE (c:TeamChatSession {id: toInteger(row[1])})
MERGE (i:ChatItem {id: toInteger(row[2])})
MERGE (u)-[:CreateChat {timeStamp: toFloat(row[3])}]->(i)
MERGE (i)-[:PartOf {timeStamp: toFloat(row[3])}]->(c);
```

- iii) Present a screenshot of some part of the graph you have generated. The graphs must include clearly visible examples of most node and edge types. Below are two acceptable examples. The first example is rendered in the default Neo4j distribution, the second has had some nodes moved to expose the edges more clearly. Both include examples of most node and edge types.



Finding the longest conversation chain and its participants

Report the results including the length of the conversation (path length) and how many unique users were part of the conversation chain. Describe your steps. Write the query that produces the correct answer.

```
MATCH p=(start:ChatItem)-[:ResponseTo*]->(end:ChatItem)
WHERE NOT ()-[:ResponseTo]->(start) AND NOT (end)-[:ResponseTo]->()
WITH p, length(p) as PathLength
ORDER BY PathLength DESC LIMIT 1
UNWIND nodes(p) AS chatItem
MATCH (u:User)-[:CreateChat]->(chatItem)
RETURN PathLength, PathLength + 1 AS ChatItemsInvolved, count(distinct u) AS UniqueUsers;
```

Interpretation & Results: The query traverses all ResponseTo paths, strictly finding roots (start) that have no inbound replies and leaves (end) that reply to nothing else. It calculates path length, sorts descending to find the longest chain, and then unwinds those nodes to distinctly count the authors of those specific items.

- **Longest Path Length:** 9
- **Number of chat items involved:** 10
- **Number of unique participating users:** 5

Identifying long conversation chains highlights sticky internal discussions or complex project bottlenecks. If 10 consecutive replies exist across 5 users, that specific topic likely generated significant organizational friction or deep collaboration that might warrant a formal meeting rather than endless text chatter.

Analyzing the relationship between top 10 chattiest users and top 10 chattiest teams

Describe your steps from Question 2. In the process, create the following two tables. You only need to include the top 3 for each table. Identify and report whether any of the chattiest users were part of any of the chattiest teams.

Chattiest Users

Users	Number of Chats
-------	-----------------

394	115
2067	111
1087	109

Chattiest Teams

Teams	Number of Chats
82	1324
185	1036
112	957

Finally, present your answer, i.e. whether or not any of the chattiest users are part of any of the chattiest teams.

User/Team Overlap Evaluation: To determine overlap between the two top-10 lists, we explicitly checked if any top 10 user belonged to a top 10 team (defined as the user having a CreatesSession or Joins relationship to a session owned by that team).

Yes, there is overlap. Upon calculating the full top ten lists, we found that User **999** (the 7th chattiest user with 105 chats) directly participated in sessions belonging to Team **52** (the 7th chattiest team with 788 chats). This demonstrates that while high chat volume isn't uniformly distributed, hyper-active teams occasionally cultivate hyper-active individual participants. Identifying this overlap enables management to identify their primary communication "hubs" or internal evangelists.

How Active Are Groups of Users?

Describe your steps for performing this analysis. Be as clear, concise, and as brief as possible. Finally, report the top 3 most active users in the table below.

To evaluate network closeness, we derived a new abstract relationship (InteractsWith) linking users directly to each other based on Mentions and Responses, executing the following queries:

```
// Mentions
MATCH (u1:User)-[:CreateChat]->(i:ChatItem)-[:Mentioned]->(u2:User)
MERGE (u1)-[:InteractsWith]->(u2);

// Responses (Responder u1 interacts with Original Author u2)
```

```

MATCH (u1:User)-[:CreateChat]->(i1:ChatItem)-[:ResponseTo]->(i2:ChatItem)->
[:CreateChat]-(u2:User)
MERGE (u1)-[:InteractsWith]->(u2);

// Remove Self-Loops
MATCH (u:User)-[r:InteractsWith]->(u)
DELETE r;

```

Clustering Coefficient for Top 10 Chattiest Users: We define the neighborhood structurally via outgoing InteractsWith relationships. Multiple edges are implicitly ignored via the initial MERGE setup, ensuring distinct edge calculations without duplication.

```

MATCH (u:User)-[:CreateChat]->(i:ChatItem)
WITH u, count(i) AS ChatCount
ORDER BY ChatCount DESC LIMIT 10
OPTIONAL MATCH (u)-[:InteractsWith]->(neighbor:User)
WITH u, collect(DISTINCT neighbor) AS neighbors
WITH u, neighbors, size(neighbors) AS k
UNWIND (CASE WHEN k = 0 THEN [null] ELSE neighbors END) AS n1
UNWIND (CASE WHEN k = 0 THEN [null] ELSE neighbors END) AS n2
OPTIONAL MATCH (n1)-[r:InteractsWith]->(n2)
WITH u, k, count(DISTINCT r) AS distinct_edges
RETURN u.id AS UserId, CASE WHEN k < 2 THEN 0.0 ELSE toFloat(distinct_edges) /
toFloat(k * (k - 1)) END AS ClusteringCoefficient
ORDER BY ClusteringCoefficient DESC;

```

Most Active Users (based on Cluster Coefficients)

User ID	Coefficient
209	1.0
668	1.0
999	0.9464